UT02_ACT12 · Diseño responsive: interfaces adaptables con CSS

DIW · 2.º DAW · UT2 · RA2
Material teórico para el alumnado

1. De CampusHub de escritorio a una interfaz adaptable

En ACT11 utilizaste **CSS Grid** para estructurar CampusHub y **Flexbox** para distribuir elementos dentro de sus regiones. El resultado podía funcionar correctamente en un monitor, pero ¿qué ocurre cuando abrimos la misma página en una ventana estrecha?

Puede suceder que la barra lateral ocupe demasiado espacio, que las tarjetas del catálogo se compriman, que los enlaces de la cabecera no quepan o que aparezca desplazamiento horizontal. Una solución no consiste en reducirlo todo: en ocasiones debemos **cambiar la distribución** de los componentes.

El **diseño responsive** adapta la presentación al espacio disponible manteniendo el mismo documento HTML. Por ejemplo, las seis tarjetas del catálogo de CampusHub podrían mostrarse en una columna en móvil, dos en una ventana intermedia y tres en escritorio. La estructura y el contenido no cambian; cambia el CSS.

1.1. Diseño fijo, fluido y responsive

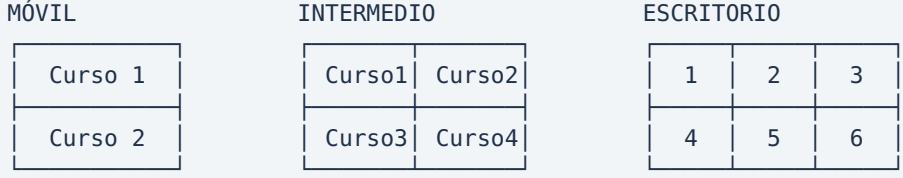
- **Fijo:** usa dimensiones rígidas, como `width: 1000px`. Puede desbordarse si hay menos espacio.
- **Fluido:** usa medidas relativas, como `width: 90%`, para aprovechar el espacio disponible.
- **Responsive:** combina medidas flexibles y reglas que reorganizan la interfaz cuando el ancho lo necesita.

```
.panel-fijo { width: 900px; }
.panel-fluido { width: 90%; max-width: 900px; }
```

`max-width` limita la anchura máxima de un elemento; no significa que tenga que alcanzar ese tamaño.

1.2. Mobile-first

Mobile-first significa definir primero una distribución funcional para pantallas estrechas y ampliarla mediante reglas adicionales. Es útil porque nos obliga a organizar primero lo esencial.



No es necesario crear una página HTML para cada dispositivo.

2. El viewport

El **viewport** es el área que el navegador utiliza para representar la página. No equivale siempre a las dimensiones físicas de la pantalla: también cambia cuando estrechamos la ventana del ordenador.

Dentro del `<head>` de nuestro HTML debemos incluir:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

- `width=device-width`: ajusta el viewport al ancho del dispositivo en píxeles CSS.
- `initial-scale=1.0`: establece la escala inicial.

Sin esta etiqueta, algunos navegadores móviles utilizan un viewport virtual ancho y reducen el contenido para mostrarlo. **La etiqueta no transforma por sí sola la página en responsive:** hace falta adaptar su CSS.

3. Media queries: reglas CSS condicionales

Una **media query** aplica determinadas declaraciones cuando se cumple una condición.

```
.aviso {
  background-color: lightblue;
  padding: 20px;
}

@media (min-width: 768px) {
  .aviso {
    background-color: lightgreen;
  }
}
```

Con menos de 768 px, el fondo es azul; a partir de 768 px, es verde. El `padding` no cambia, porque la media query no lo sobrescribe.

3.1. `min-width` frente a `max-width`

Condición	Cuándo se cumple
`(min-width: 768px)`	Cuando el viewport mide **768 px o más**
`(max-width: 768px)`	Cuando el viewport mide **768 px o menos**

Ambas condiciones son válidas, pero en esta actividad utilizaremos preferentemente `min-width` para trabajar con mobile-first.

3.2. Coincidencia de condiciones y cascada

```
.caja { background-color: orange; }

@media (min-width: 600px) {
  .caja { background-color: blue; }
}

@media (min-width: 900px) {
  .caja { background-color: green; }
}
```

Si el viewport mide 1000 px, **ambas media queries se cumplen**. Dado que los selectores tienen la misma especificidad, prevalece la última declaración aplicable: la del fondo verde. Es la cascada que ya conoces.

Predicción 1 · ¿Qué regla prevalece?

No ejecutes el código todavía. Utiliza el ejemplo anterior y responde en `respuestas.md` del laboratorio:

1. ¿Qué color tendrá `.caja` a 500 px, 750 px y 1000 px?
2. ¿Qué color tendrá exactamente a 900 px?
3. ¿Qué dos condiciones se cumplen a 1000 px y por qué se ve un único color?

Cuando hayas escrito tu hipótesis, compruébala modificando el ancho del navegador.

4. Breakpoints: cuándo reorganizar la página

Un **breakpoint** es una anchura a partir de la cual decidimos cambiar una parte del diseño. Como referencia inicial puedes utilizar:

Anchura del viewport	Presentación orientativa
Menos de 768 px	Estrecha
768–1023 px	Intermedia
Desde 1024 px	Amplia

No son medidas obligatorias. Un breakpoint debe responder a una necesidad del contenido: por ejemplo, cuando la navegación deja de caber o las tarjetas resultan ilegibles.

```
.tarjeta { padding: 12px; }

@media (min-width: 768px) {
  .tarjeta { padding: 20px; }
}

@media (min-width: 1024px) {
  .tarjeta { padding: 28px; }
}
```

No repetimos el resto de las propiedades de `.tarjeta` porque siguen siendo válidas.

5. CSS Grid responsive

El número de columnas puede cambiar sin modificar el HTML:

```
<section class="galeria">
  <article class="tarjeta">Curso 1</article>
  <article class="tarjeta">Curso 2</article>
  <article class="tarjeta">Curso 3</article>
</section>
```

```
.galeria {
  display: grid;
  grid-template-columns: 1fr;
  gap: 20px;
}

@media (min-width: 768px) {
  .galeria { grid-template-columns: repeat(2, 1fr); }
}

@media (min-width: 1024px) {
  .galeria { grid-template-columns: repeat(3, 1fr); }
}
```

En pantallas estrechas hay una columna; en pantallas intermedias dos y en pantallas amplias tres. `display: grid` y `gap` permanecen definidos una sola vez.

5.1. Tamaños rígidos y fracciones

```
/* Tres columnas de 300 px: necesitan espacio suficiente. */
.galeria-rigida { display: grid; grid-template-columns: 300px 300px 300px; gap: 20px; }

/* Reparte el espacio disponible entre tres columnas. */
.galeria-flexible { display: grid; grid-template-columns: repeat(3, 1fr); gap: 20px; }
```

La unidad `fr` facilita la distribución flexible, aunque el contenido puede imponer tamaños mínimos. Recuerda los problemas de tamaño intrínseco trabajados en ACT11 y el uso de `minmax(0, 1fr)` cuando proceda.

También podemos modificar `grid-template-areas` en media queries para reorganizar cabecera, lateral, contenido y pie sin cambiar su orden HTML. La distribución visual no debe dificultar el orden lógico de lectura.

6. Flexbox responsive

Flexbox resulta útil para adaptar una navegación o los controles de un componente:

```
.menu {
  display: flex;
  flex-direction: column;
  gap: 12px;
}

@media (min-width: 768px) {
  .menu { flex-direction: row; }
}
```

Los enlaces aparecen en columna en móvil y en fila cuando hay espacio. Según las necesidades también puedes utilizar `flex-wrap`, `align-items` y `justify-content`, propiedades conocidas de ACT10.

Grid y Flexbox no se sustituyen entre sí: Grid es útil para regiones, filas y columnas; Flexbox para alinear elementos a lo largo de un eje. Una página responsive puede utilizar ambos a la vez.

7. Dimensiones adaptables e imágenes

```
.panel {
  width: 90%;
  max-width: 900px;
}

.imagen {
  max-width: 100%;
  height: auto;
}
```

`max-width: 100%` evita que una imagen supere el ancho de su contenedor; `height: auto` conserva sus proporciones. Si necesitas llenar una caja de tamaño determinado, puedes recuperar `object-fit`, visto en la auditoría anterior.

Distingue estos usos:

- `max-width: 900px;` dentro de un selector limita el ancho de un **elemento**.
- `@media (max-width: 900px)` comprueba el ancho del **viewport**.

Cuando aparezca desplazamiento horizontal, investiga su causa: anchuras rígidas, pistas que no caben, contenido largo, imágenes o separaciones excesivas. No ocultes el problema arbitrariamente con `overflow: hidden`.

Predicción 2 · Cálculo de espacio en Grid

Sin ejecutar código, analiza:

```
.galeria {
  display: grid;
  grid-template-columns: 300px 300px 300px;
  gap: 20px;
}
```

1. ¿Cuánto ocupan las tres columnas juntas?
2. ¿Cuántos huecos hay entre las columnas y cuánto suman?

3. ¿Cuál es el ancho mínimo necesario para mantenerlas sin desbordamiento, sin padding ni bordes?
4. ¿Qué ocurrirá si el contenedor solo tiene 700 px?
5. ¿Qué cambio de distribución podrías hacer en una media query?

Escribe tu respuesta **antes de probarla** con el navegador.

8. Comprobación con Chrome DevTools

1. Abre la página mediante Live Server.
2. Abre DevTools (`F12` en Windows; también puedes usar el menú del navegador).
3. Activa el modo de dispositivos.
4. Selecciona *Responsive* y modifica manualmente el ancho.
5. Comprueba la presentación no solo en los breakpoints, sino también **entre ellos**.

Verifica que la navegación sigue siendo utilizable, el texto resulta legible, las tarjetas no se solapan y no aparece desplazamiento horizontal innecesario. Comprueba también el comportamiento de las imágenes y la sección adicional de CampusHub.

9. Aplicación de lo aprendido

Realiza los laboratorios incluidos en `laboratorios/`. Después desarrolla el **reto autónomo de CampusHub** descrito en `ENUNCIADOS.md`, trabajando sobre tu proyecto terminado de ACT11.

La nueva versión debe conservar el HTML y contenido del proyecto anterior siempre que sea posible: el reto consiste en **adaptar el CSS y razonar por qué cambian las distribuciones**.